

Chapter 6 --- SQL Data Types

Every column in a table must be given a data type. A data type tells MySQL what kind of value a column can hold (a number, text, a date, and so on), how much storage it needs, and what operations are valid on it. Choosing the right data type keeps your database fast, accurate, and free of wasted storage.

Definition: A data type is a rule that defines the kind of data a column can store, the range of values it accepts, and the amount of memory it occupies.

6.1 Working With Numeric Data Types

Numeric types store numbers --- either whole numbers (integers) or numbers with decimal points. MySQL offers several integer sizes so you only pay for the storage you actually need.

Integer Types

Type Storage Approx. Range Typical Use
(Signed)

TINYINT 1 byte -128 to 127 Small flags, age, small counters

SMALLINT 2 bytes -32,768 to 32,767 Year, small quantity fields

MEDIUMINT 3 bytes -8,388,608 to 8,388,607 Medium-range counts

INT / INTEGER 4 bytes -2.1 billion to 2.1 billion IDs, general whole numbers

BIGINT 8 bytes -9.2 quintillion to 9.2 quintillion Very large counters, big IDs

UNSIGNED: Any integer type can be marked UNSIGNED to store only non-negative numbers, which doubles the positive range. Example:

Age TINYINT UNSIGNED -- stores 0 to 255 instead of -128 to 127

Decimal (Fixed-Point) Types

Used when exact precision matters --- money, for example, must never be rounded incorrectly.

Type Description Example

DECIMAL(M, D) Exact fixed-point DECIMAL(8,2) stores up
number. M = total to 999999.99
digits, D = digits
after the decimal
point.

NUMERIC(M, D) Identical to DECIMAL in NUMERIC(10,2)
MySQL --- just another
name for it.

Price DECIMAL(8,2) -- e.g. 45999.99

Floating-Point Types

Used for scientific or approximate values where tiny rounding
differences are acceptable --- never use these for money.

Type Storage Precision

FLOAT 4 bytes Approximate, ~7
significant digits

DOUBLE 8 bytes Approximate, ~15
significant digits
(more accurate than
FLOAT)

■ Advantage of Choosing the Right Numeric Type Using TINYINT instead
of INT for a column that only ever stores 0--150 (like age) saves 3
bytes per row. Across millions of rows that adds up to real disk-space
and memory savings, and smaller columns are also faster to index and
scan.

6.2 Working With CHAR, VARCHAR and TEXT Data Types

These types store text. The key difference between them is how the
storage size is decided and how MySQL handles the actual length of the
text.

CHAR(n) --- Fixed-Length String

Always reserves exactly n characters, no matter how short the actual
text is. MySQL pads shorter values with spaces internally (and removes
trailing spaces on retrieval).

Gender CHAR(1) -- always stores exactly 1 character, e.g. 'M' or 'F' StateCode CHAR(2) -- e.g. 'KA', 'TX'

Best for values that are always the same length --- gender codes, fixed
country codes, fixed-length IDs.

VARCHAR(n) --- Variable-Length String

Stores only as many characters as are actually entered, up to a maximum of n. It uses 1-2 extra bytes to record the actual length, so it is more storage-efficient for text that varies in length.

StudentName VARCHAR(50) -- stores 'Aarav' using only 5 characters + length bytes

Best for names, emails, addresses, city names --- anything whose length genuinely varies.

TEXT --- Large Variable-Length Text

Used for long text that can exceed VARCHAR's practical limits --- paragraphs, descriptions, comments, articles.

Type Maximum Size Typical Use

TINYTEXT 255 bytes Very short notes

TEXT 65,535 bytes (~64 KB) Comments, descriptions

MEDIUMTEXT 16 MB Articles, long descriptions

LONGTEXT 4 GB Very large documents

CHAR vs VARCHAR vs TEXT --- Comparison

Feature CHAR VARCHAR TEXT

Length Fixed Variable (up to a Variable, much limit) larger

Storage used Always n Actual length + Actual length + characters 1-2 bytes overhead

Speed Slightly faster Efficient for Slower; stored for fixed data typical text separately from the row for large values

Can be indexed Yes Yes Only a prefix, by fully default

Example use CHAR(2) state VARCHAR(100) TEXT blog post code email body

■ Advantage of Choosing Correctly Picking VARCHAR over TEXT for short fields like names or emails keeps the row itself compact and lets MySQL index and sort the column efficiently. Reserve TEXT only for genuinely long content --- using it everywhere "just in case" makes queries slower and indexing harder.

6.3 Working With Date, Time, and Boolean Values

Date and Time Types

Type Format Range / Notes

DATE YYYY-MM-DD Stores only the date,
e.g. 2026-09-10

TIME HH:MM:SS Stores only the time of
day, e.g. 14:30:00

DATETIME YYYY-MM-DD HH:MM:SS Stores date and time
together; not affected
by time zone

TIMESTAMP YYYY-MM-DD HH:MM:SS Like DATETIME, but
stored in UTC and
auto-converted to the
connection's time zone;
smaller range
(1970--2038)

YEAR YYYY Stores just a 4-digit
year, e.g. 2026

Example table using date/time columns:

```
CREATE TABLE Attendance ( AttendanceID INT PRIMARY KEY, StudentID INT, AttendDate DATE, CheckInTime
```

DATETIME vs TIMESTAMP: use DATETIME for fixed, timezone-independent
values (like a date of birth or a scheduled exam date). Use TIMESTAMP
for values that should reflect "when something actually happened" (like
a row's creation or last-update time), since it can auto-update and
adjusts for time zone.

Boolean Values

MySQL does not have a true, separate BOOLEAN storage type. BOOLEAN (or
BOOL) is simply an alias for TINYINT(1) --- a value of 0 is treated as
FALSE, and any non-zero value (typically 1) is treated as TRUE.

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY, IsActive BOOLEAN DEFAULT TRUE -- stored internally as
```

Good to know for interviews: writing BOOLEAN in MySQL is only for
readability --- under the hood MySQL stores and treats it exactly like
TINYINT(1).

6.4 Choosing Appropriate Data Types for Application Data

Picking the right data type is not just a technical formality --- it
directly affects storage cost, query speed, and data accuracy. A few
practical guidelines:

- Match the type to the real-world range. Don't use INT for a column

that will only ever hold 0-5 (like a rating) --- TINYINT is enough.

- Use DECIMAL for money, never FLOAT/DOUBLE. Floating-point types can introduce tiny rounding errors that are unacceptable in financial data.

- Prefer VARCHAR over CHAR for variable-length text such as names and addresses; reserve CHAR for genuinely fixed-length codes.

- Use TEXT only when content can genuinely be long. For short fields, VARCHAR is faster to index and search.

- Use DATE/DATETIME/TIMESTAMP instead of storing dates as text. This allows correct sorting, date arithmetic, and validation that a plain string column cannot give you.

- Use BOOLEAN (TINYINT(1)) for true/false flags instead of storing 'Yes'/'No' as text --- it's smaller and faster to compare.

- Avoid over-sized columns "just in case". A VARCHAR(255) for every text field wastes planning effort --- size columns to realistic maximum lengths.

Worked Example --- Choosing Types for a Student Table

Column Chosen Type Reason

StudentID INT (PRIMARY KEY) Whole number, comfortably fits typical ID ranges

StudentName VARCHAR(50) Text length varies from student to student

Gender CHAR(1) Always exactly one character ('M'/'F'/'O')

DateOfBirth DATE Only the date is needed, no time component

FeesPaid DECIMAL(10,2) Money --- must be exact, never approximate

IsActive BOOLEAN Simple true/false flag

Bio TEXT Free-form text that can be long and of unknown length

■ Final TakeawayA data type is a contract between you and MySQL about

what a column can hold. Choose numeric types by realistic range, use DECIMAL for money, VARCHAR for typical text, TEXT only for long content, proper DATE/TIME types instead of plain strings, and BOOLEAN for flags. Good choices here make your database smaller, faster, and far less error-prone.

Interview Questions — Chapter 6

Q1. DECIMAL vs DOUBLE for money?

Ideal answer: Use DECIMAL for exact financial values. FLOAT and DOUBLE are approximate floating-point types and can introduce rounding differences.

Q2. CHAR vs VARCHAR?

Ideal answer: CHAR is fixed-length; VARCHAR stores variable-length text up to its declared maximum.

Q3. DATETIME vs TIMESTAMP?

Ideal answer: DATETIME is suitable for fixed date/time values without timezone conversion; TIMESTAMP is commonly used for event timestamps and has timezone conversion behavior.

Q4. Why should dates not be stored as VARCHAR?

Ideal answer: Proper date/time types support validation, chronological comparison, sorting, and date arithmetic.

Q5. Why choose the smallest practical numeric type?

Ideal answer: It reduces storage and can improve index and scan efficiency, especially across large tables.